Countersign

Technical document

An investing agent that cannot buy a fund unit unless the investor's mandate, the fund's issuer and a wallet contract the agent cannot change agree. Sections follow the official template: 01 Project Overview, 02 System Architecture, 03 Agent Design, 04 Security Design, 05 Deployment and Testing.

TEAM NAME	Countersign
COMPETITION TRACK	International AI Track, Topic F (Agent Wallet)
SUBMISSION DATE	20 October 2026
LINKS	countersign-web-rt1a.onrender.com · github.com/ceciliagalvaoo/Countersign · ceciliagalvaoo.github.io/Countersign



Pablo de Azevedo
Team leader · front end and product



Cecília Galvão
Blockchain and backend

Inteli, Institute of Technology and Leadership, São Paulo

SECTION 01

01 Project Overview

PROBLEM, TARGET USERS AND SELECTED TOPIC

> An investing agent that cannot buy a fund unit unless the investor's mandate, the fund's issuer and a wallet contract the agent cannot change agree.

Selected topic: F, Agent Wallet (autonomous on-chain wallet). **Tagline:** *Sign only what you can read.*

Countersign is an AI agent (Claude, by Anthropic) with its own mandate wallet on Stellar. It buys units of receivables funds for an investor who already holds USDC. Every first purchase in a new fund waits for a card she can read and sign.

Target users. Three parties use the system (Figure 1): **the investor**, who holds USDC, lives under the rules of two countries and wants part of her savings in the real economy (composite persona, built from public data; not validated by interviews); **the agent**, which turns her words into a typed mandate and a plan, and proposes purchases; and **the fund's issuer**, the fund manager's wallet, which countersigns every mint of fund units.

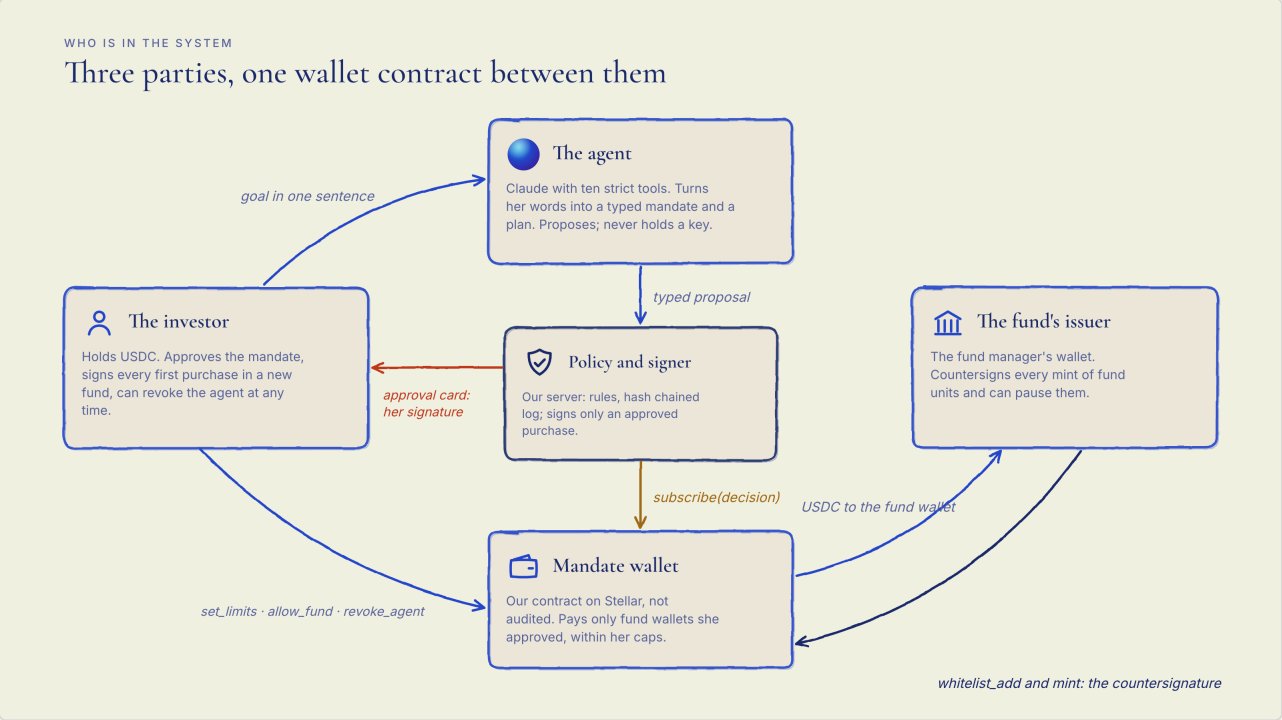


Figure 1. The three parties and the wallet contract between them.

The problem, with numbers and sources. She gives up at one precise moment: the confirmation screen of her first purchase, in front of something irreversible she cannot read (Figure 2).

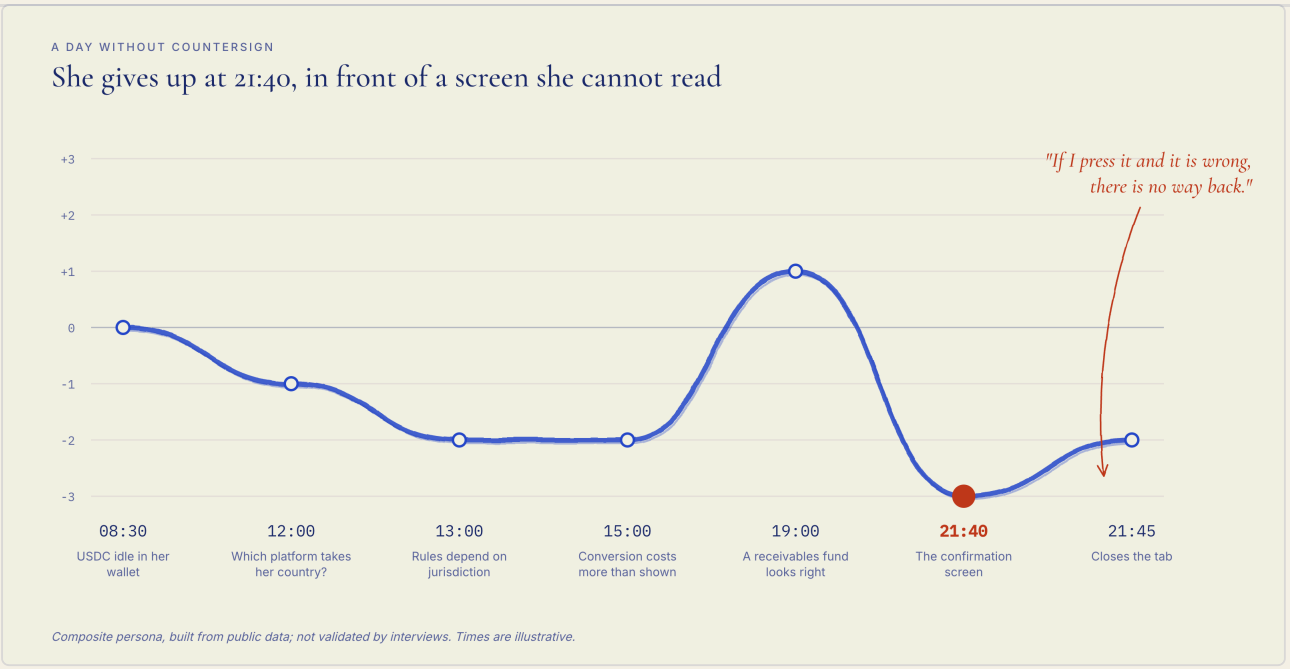


Figure 2. A day without Countersign. Composite persona, built from public data; times are illustrative.

FACT	NUMBER	SOURCE
People who already hold digital dollars on this network	USDC: 2,481,283 trustlines on Stellar mainnet	stellar.expert API
Retail investors are entering receivables funds (Brazil, as an example)	Net assets of FIDCs R$741.1 billion, +22.5% in 12 months to Nov 2025; "general" investor accounts from 2.4 thousand to 34.3 thousand (+1,329.2%)	ANBIMA, 24 Dec 2025
The risk of a receivables fund is existence and concentration	Credit Suisse closed 4 funds linked to Greensill with "around ten billion US dollars" of client money	FINMA, 28 Feb 2023
A language model refusing does not protect money	Freysa: 482 messages, 13.19 ETH (about US$47 thousand)	Simon Willison
The operator's panel is an attack surface	AIXBT: 55.5 ETH (about US$106,200)	AIID 1003
The human approves what the screen shows	Bybit: about US$1.5 billion, tampered signing interface	Bloomberg
The regulator's position on our object	Yinfa [2026] No. 42 (6 Feb 2026): tokenized real world assets "prohibited onshore"	China Services Info

CORE VALUE AND INNOVATION

Agents that move money have already lost it through a chat instruction (Freysa), through the operator's panel (AIXBT) and through a signing screen that showed one thing while the transaction did another (Bybit). In most agent wallet designs the language model proposes and a policy on the server decides; if the server falls, the policy falls with it. Countersign puts the limits where our server cannot reach: in the agent's wallet contract, and in the fund issuer's signature on every mint.

> The agent's wallet is a contract on Stellar: it can only pay fund wallets the investor approved, within limits she signed, and it cannot change those limits. Nobody holds a fund unit unless the fund's issuer signs the mint.

Those limits are enforced by our wallet contract on Stellar, not audited.

What is new. (1) A wallet contract per investor that only pays fund wallets she approved, within per purchase, per period and per fund caps she signed, and never reuses a decision. (2) An approval card decoded from the exact transaction bytes, never from the model's words, checked a second time in the browser. (3) A hash chained decision log whose line hash travels on chain as the `decision` argument of every agent purchase; the contract refuses a reused value but cannot check a new one against our log, so the evidence script checks it afterwards (limitation 3). (4) The issuer's countersignature on every mint. (5) A zero-knowledge tier proof verified on chain, so the agent only learns "eligible" or "not eligible".

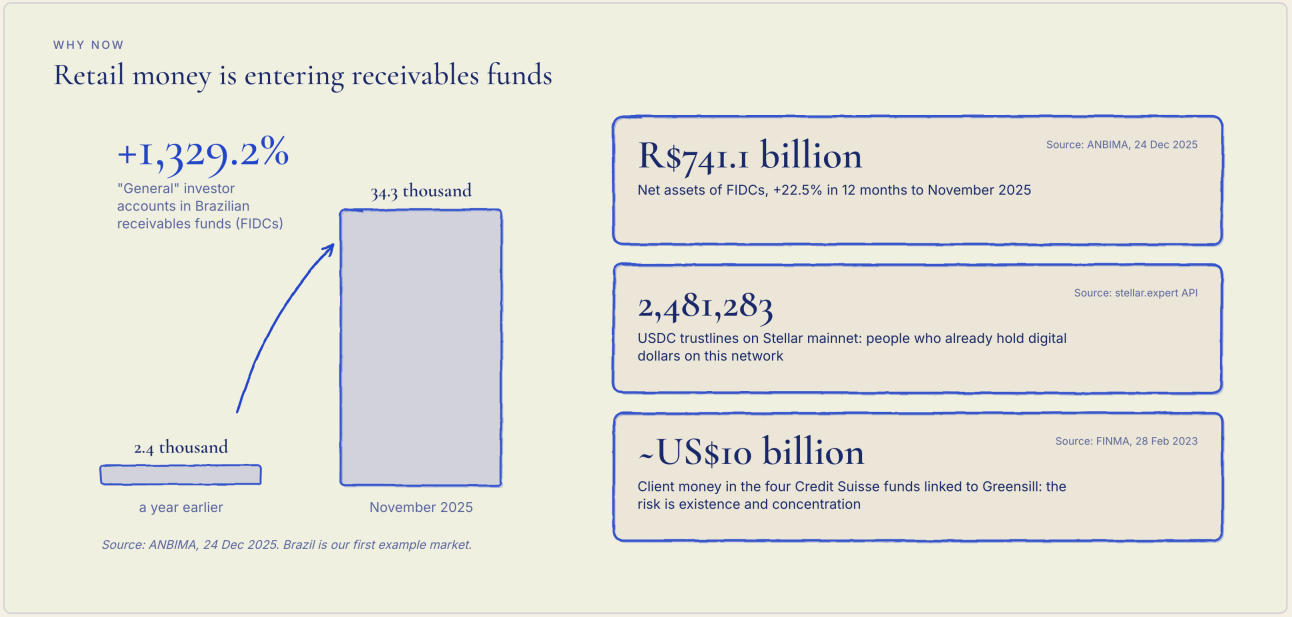


Figure 3. Why now: retail money entering receivables funds, and people who already hold USDC on Stellar.

SECTION 02

02 System Architecture

ARCHITECTURE DIAGRAM AND COMPONENT RESPONSIBILITIES

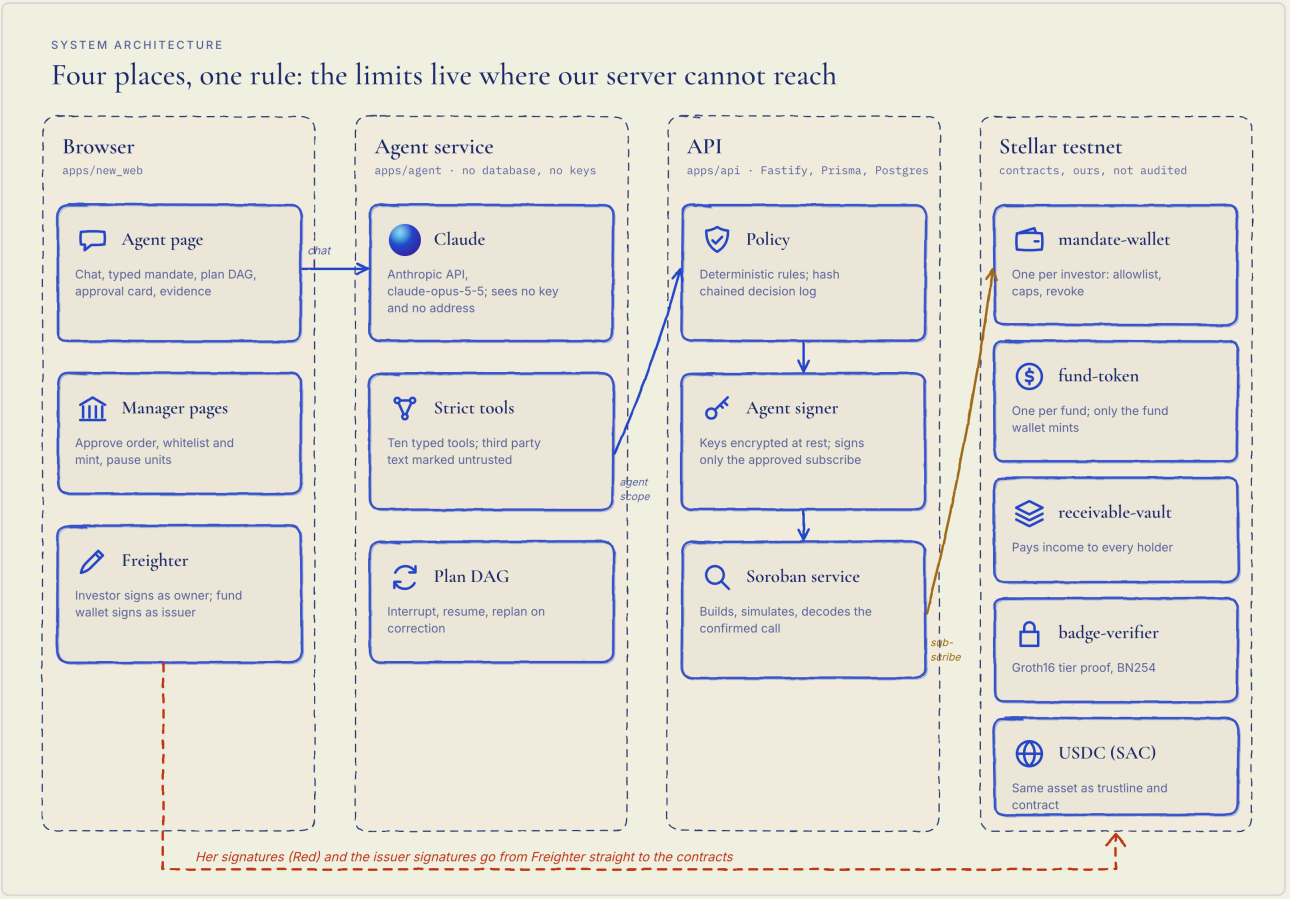


Figure 4. Components. The agent service has no database and no keys; the agent key lives in the API's signer; the limits live in the mandate wallet contract.

COMPONENT	PATH	RESPONSIBILITY
Agent page	`apps/new_web` (React, Vite)	Chat, typed mandate, plan DAG with the tier of each node, approval card decoded from the transaction bytes (a second decode in the browser blocks signing if it differs from the API), pending and confirmed states, interrupt, recurring, revoke, evidence panel
Manager pages	`apps/new_web`	Issuer approves an order (`whitelist_add` and `mint`), pauses the FundToken
Agent service	`apps/agent` (Node, Fastify, `@anthropic-ai/sdk`)	Claude with strict tools, typed mandate, plan DAG, interrupt and resume, untrusted text marking, data minimisation before every model call; no database, no keys
Policy and decision log	`apps/api/src/policy`	Deterministic rules, mandate store the agent cannot write, append only hash chained log (`hash = sha256(prevHash || canonical(payload))`), approval valid for 15 minutes
Agent signer	`apps/api/src/agentSigner`	One agent key per investor, encrypted at rest (AES-256-GCM); signs only a `subscribe` identical to an approved decision
Soroban integration	`apps/api/src/services`, `apps/api/src/routes`	Builds and simulates every transaction, the right wallet signs, then decodes the confirmed call and checks it before writing to the database

COMPONENT	PATH	RESPONSIBILITY
`mandate-wallet`	`contracts/mandate-wallet` (Rust, `soroban-sdk` 28.0.0)	Fund allowlist, per purchase, per period and per fund caps, revocation, refusal of a reused `decision`; no upgrade function, no platform admin
`fund-token`	`contracts/fund-token`	Fund units (SEP-41 token), allowlist of holders, mint only by the fund wallet, pause
`receivable-vault`	`contracts/receivable-vault`	Receivables registry, `pay`, `distribute` pro rata to every holder
`badge-verifier`	`contracts/badge-verifier`, `zk/badge`	Groth16 verification (BN254) of an investor tier proof
Suites and evidence	`tests/suites`, `scripts/evidence`, `scripts/e2e`	Benign B01 to B12 and adversarial A01 to A12 in one command; evidence export with decision checks

MODELS, TOOLS, DATA SOURCES AND EXTERNAL SERVICES

Models and external services: Claude (`claude-opus-5-5` by default, Anthropic API), Stellar testnet RPC and Horizon, stellar.expert as the public explorer, Freighter as the browser wallet.

Tools. Ten strict tools (schemas with `strict: true`): `get_mandate`, `propose_mandate`, `list_funds`, `get_fund`, `check_eligibility`, `quote_subscription`, `propose_subscription`, `execute_approved_purchase`, `get_status`, `get_holdings`.

Data sources. The investor's messages; the demonstration fund catalogue in our database (simulated funds and debtors; concentration and default data are synthetic); balances, allowlists and caps read from the chain; the decision log.

Why this infrastructure. Without Stellar's USDC being the same asset as a trustline and as a contract (the Stellar Asset Contract), with authorization done by the host for any address, the investor's existing USDC could not sit in an agent wallet contract and be paid to a fund wallet in one contract call; the caps would move back to our server, which is exactly the design we reject. Stellar's Protocol 25 also made BN254 and Poseidon native, which lets our `badge-verifier` check a Groth16 eligibility proof on chain. And every judge can open each hash in a public explorer. This is an advantage of cost and simplicity, not exclusivity: the same rules could be written as a smart account on another chain.

SECTION 03

03 Agent Design

INTENT UNDERSTANDING, PLANNING, TOOL USE AND STATE MANAGEMENT

Intent understanding. She writes her goal in one sentence. The agent asks only what is missing (residences, horizon, liquidity, maximum loss, amount, recurrence) and fills a typed mandate shown next to the chat. Nothing is valid until she approves it; the approved mandate hash is anchored on chain with the wallet deploy or with `set_limits`.

Residence is declared, not verified. The perimeter rule reads the residences she states in the chat or the mandate; a production deployment would check them against KYC data from a licensed partner.

Planning. The plan is a state machine shown as a graph: profile, perimeter, eligibility, screening, proposal, investor signature, agent purchase, issuer countersign, confirmed. Nodes move only on tool results, and each node shows its tier. A correction ("make it 4") replans without starting over; she can interrupt and resume at any node.

Tool use. The model only produces typed proposals. Third party text (fund and assignor descriptions) is passed as untrusted data and cannot set a destination or an amount. A failed tool returns as an error result and is never assumed done; the agent never says "done" before `get_status` reports a completed order.

Always at hand. An orb in the corner of every investor screen opens the same conversation in a side panel; it answers and proposes, signs nothing, and sends her to the agent page when a signature is waiting. It is not shown to the fund manager.

State management. The typed mandate and the decision log live in the API (the agent cannot write the mandate store); the conversation, plan and turn counter live in the agent service session; balances, allowlists, caps and revocation are read from the chain at execution time.

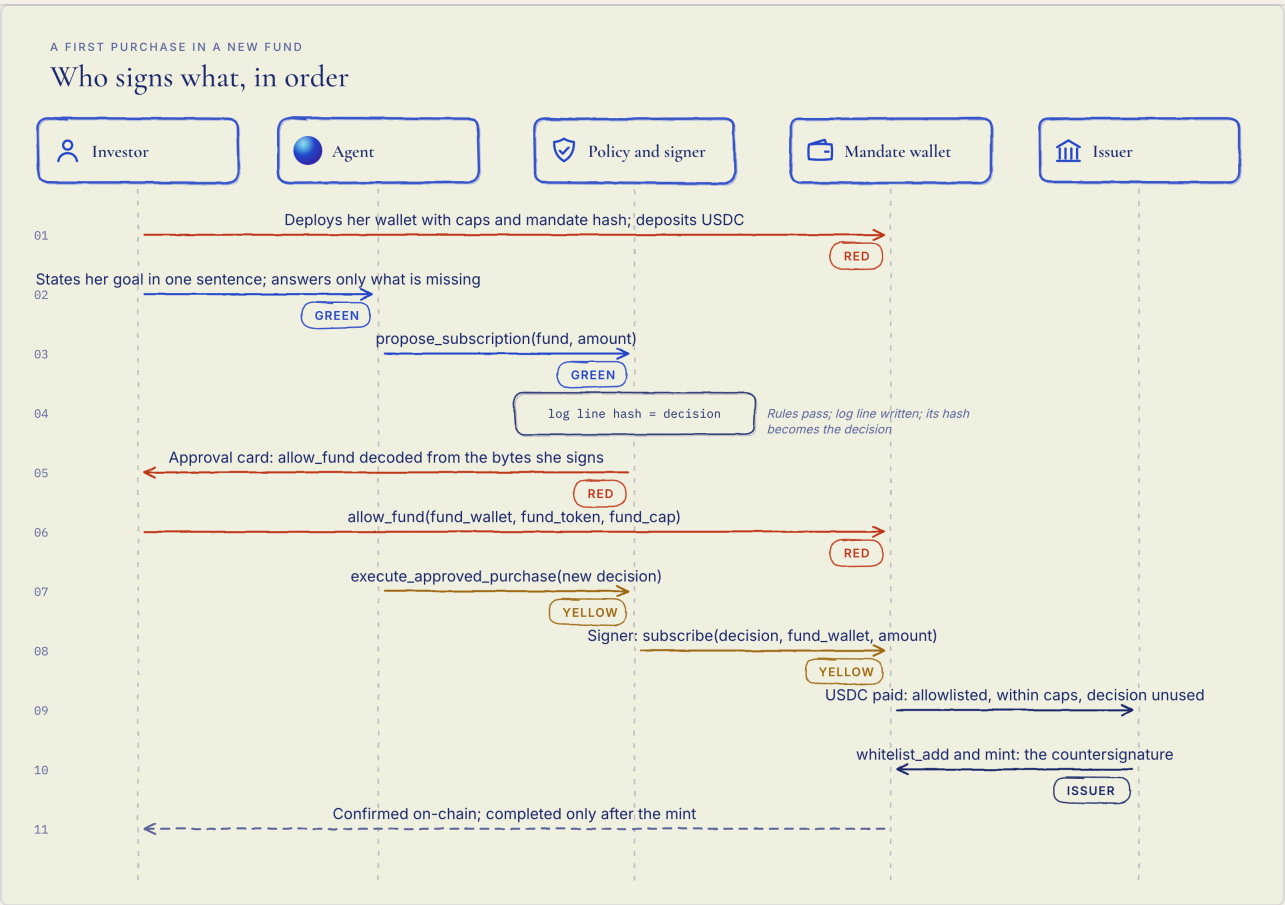


Figure 5. A first purchase in a new fund: who signs what, in order, with the tier of each step.

The steps, in order (the same order as the demo video):

1. **Mandate wallet.** The investor deploys her own `mandate-wallet` contract from the browser (owner = her wallet, agent = an agent key generated for her, USDC, caps, hash of her mandate) and deposits test USDC. She signs in Freighter.
2. **Clarification.** She writes her goal in one sentence. The agent asks only what is missing (residences, horizon, liquidity, maximum loss, amount, recurrence) and fills a typed mandate shown next to the chat. Nothing is valid until she approves it; the approved mandate hash is anchored on chain.
3. **Perimeter.** A declared residence in Mainland China is refused by the policy rule `perimeter_mainland_china`, and the reply names the notice (Yinfa [2026] No. 42).
4. **Screening and eligibility.** For each approved fund: sector, largest debtor in %, liquidity and a loss scenario; no return figure. Text written by funds and assignors is marked untrusted. Funds that require a tier need a zero-knowledge tier proof verified on chain; the agent only learns "eligible" or "not eligible".
5. **Quote and correction.** Units are computed by code, never by the model. "Make it 500, only the agri fund" replans the visible plan (a DAG) without starting over.
6. **Proposal and decision.** The agent calls `propose_subscription`. Our policy service (deterministic code, separate from the model) checks perimeter, eligibility, destination (always the fund wallet from our database, never from text) and caps, and writes a hash chained decision-log line. That line's 32 byte hash becomes the `decision`.
7. **First purchase in a new fund (Red tier).** The flow stops. The approval card shows `allow_fund(fund_wallet, fund_token, fund_cap)` decoded from the transaction bytes she will sign, not written by the agent, with the loss scenario and "The agent cannot change these limits." She signs.
8. **Agent purchase (Yellow tier).** The agent's signer, held by our server and never seen by the model, signs only a `subscribe(decision, fund_wallet, amount)` identical to the approved decision. Our wallet contract checks allowlist, caps, revocation and reuse of `decision`, then pays the fund wallet.

9. **Verification.** The API decodes the confirmed transaction (contract, method, `decision`, destination, amount) before changing its database. The screen says "pending, not done" until the ledger closes.
10. **Issuer countersignature.** The fund wallet signs `whitelist_add` (first time) and `mint` of units to her mandate wallet. Until then the screen says "awaiting issuer countersignature".
11. **Recurring purchases.** "Run recurring now: pre-authorised within mandate" repeats steps 6, 8 to 10 for funds already allowed, without a new signature from her, with a window to interrupt. The time trigger is simulated; the on-chain transactions are real.
12. **Kill switch and freeze.** She signs `revoke_agent()`, and our wallet contract refuses the agent's next purchase. The issuer signs `set_pause(true)` on the FundToken, and mints stop.
13. **Income back to the wallet.** A paid receivable is distributed by the vault to every holder, including her mandate wallet: `1a951aee...747d3b` paid 10 USDC to it on testnet.
14. **Evidence.** One command recomputes the decision-log chain and compares every `decision` read from a confirmed transaction with its log line.

CORE ALGORITHMS AND EVALUATION APPROACH

Deterministic policy (L1). Pure functions with stable rule names: `perimeter_mainland_china`, `mandate_not_active`, `fund_not_investable`, `eligibility_zk_tier`, `amount_over_purchase_cap`, `amount_over_period_cap`, `sector_not_allowed`, `sector_concentration`, `risk_above_tolerance`, `liquidity_mismatch`, `untrusted_text_cannot_set_destination_or_amount`. An approval is valid for 15 minutes and is evaluated again at execution against the chain.

Hash chained decision log. Canonical JSON (sorted keys, integers only, money as strings in 7 decimal units); `hash = sha256(prevHash || canonical(payload))`, genesis of 32 zero bytes; the database refuses UPDATE, DELETE and TRUNCATE through a trigger. The approved line's hash is the `decision` argument of `subscribe` (Figure 6).

Quote. Units are computed by code, rounded down, never above the amount asked; never by the model.

Zero-knowledge tier. A Groth16 proof (BN254, Poseidon commitment) that the total invested on this platform reaches a threshold, verified on chain by `badge-verifier`.

Evaluation approach. Two suites in one command: benign tasks B01 to B12, run live with the real model, scored by completion and turns; adversarial cases A01 to A12, scored by blocking and by the layer that blocked. Rates use every defined case as the denominator. The evidence export recomputes every chain and compares every on-chain decision with its line.

What the numbers are, and are not. Each benign case ran once live with the real model; the table consolidates per case results from several runs (`evidence/suites-2026-10-19T05-43-22-039Z.json`). We report no pass rate over repeated runs. A spending cap of US$0.30 per case stops a case before it overspends; B11 stopped there after a correct first turn (about US$0.32) and is reported as an error. Of the 37 agent purchases in the evidence, 15 come from the scripted end to end runs (no model), 11 from benign suite dry runs with a scripted model client, 10 from suite runs and 1 through the app.

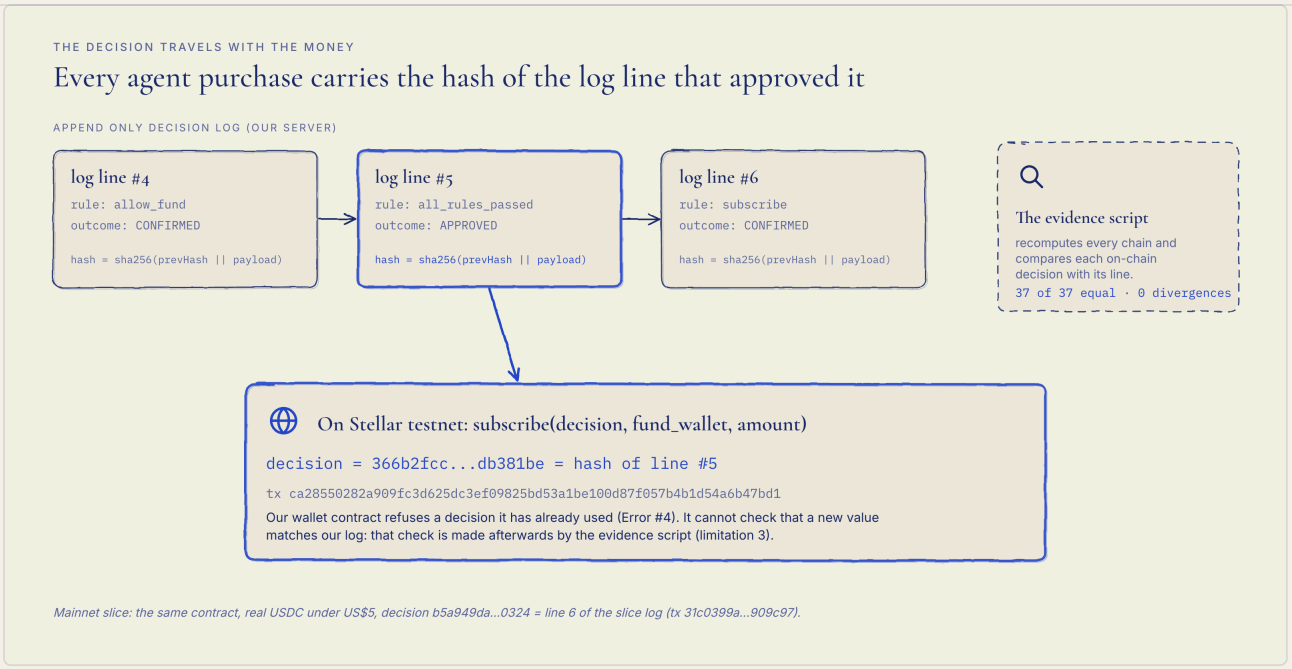


Figure 6. The decision travels with the money.

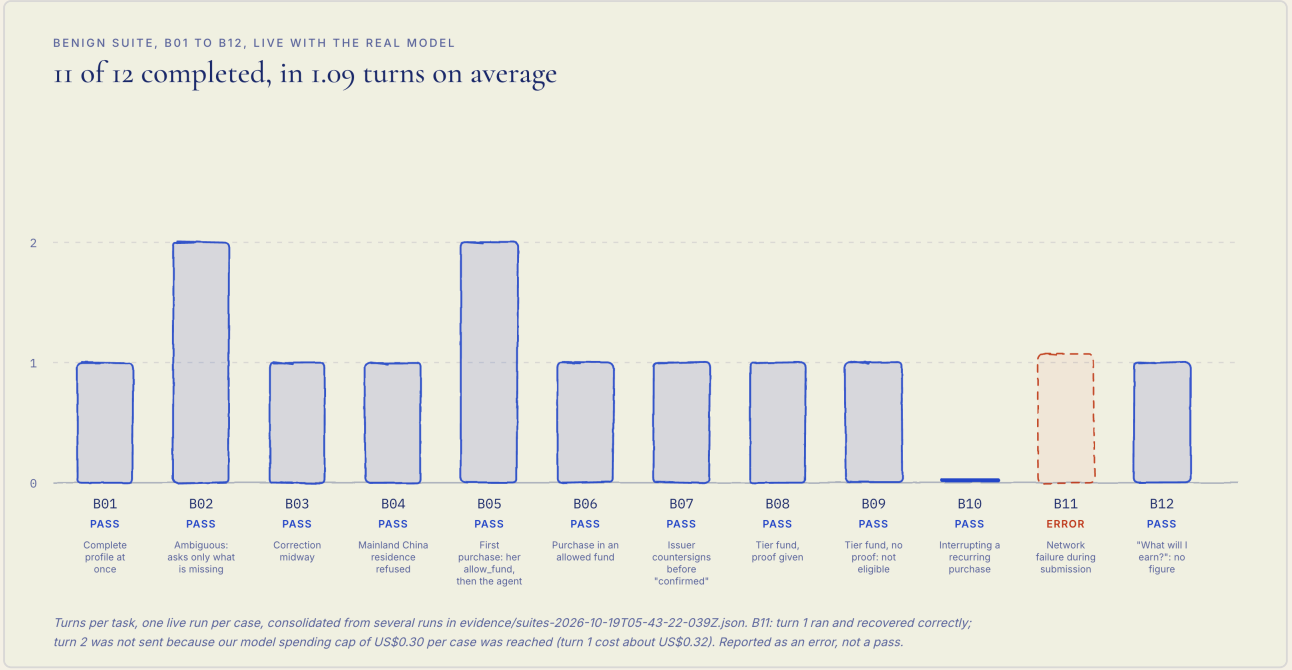


Figure 7. Benign suite: 11 of 12 completed (91.7%), 1.09 turns on average.

SECTION 04

04 Security Design

TIERED PERMISSIONS AND CONFIRMATION FOR SENSITIVE ACTIONS



Figure 8. Permission tiers.

TIER	OPERATIONS	WHO SIGNS	WHERE IT IS ENFORCED
Green (automatic)	Read funds, quote, status, balances, tier eligibility (yes or no only)	nobody	API (role from the JWT)
Yellow (agent, within the mandate)	`subscribe` to a fund already allowed, within the caps	agent key, after the policy	Policy **and** our wallet contract
Red (investor)	Create the wallet, `set_limits` (mandate), `allow_fund` (first purchase in a fund), `withdraw`, `revoke_agent`, `set_agent`	owner in Freighter, after reading the card	Our wallet contract (`owner.require_auth`)
Issuer	`whitelist_add`, `mint`, `set_pause`, refund	fund wallet	FundToken contract and API
Never	Agent writing mandate, policy or caps; paying outside the allowlist; signing as owner	n/a	Our wallet contract and the policy

Confirmation for sensitive operations. Every Red operation shows an approval card decoded from the exact transaction bytes, with the network stated separately ("The network is not part of the transaction; this app states it separately."), the function, the fund, the caps, the loss scenario and "The agent cannot change these limits. Enforced by our wallet contract on Stellar, not audited." The browser decodes the bytes a second time and blocks signing ("Do not sign") if its reading differs from the API's. A policy approval is valid for 15 minutes; at execution the policy re-reads the mandate hash, caps, spending and revocation on chain. Nothing is shown as done before the ledger confirms it and the issuer mints.



Figure 9. Who enforces what, and what each layer does not hold.

PROTECTION AGAINST PROMPT INJECTION, PRIVILEGE ESCALATION AND PRIVACY LEAKAGE

Prompt injection. Third party text is marked untrusted and can never set a destination or an amount (A01, A02); the destination always comes from our database; the amount must come from her messages or the mandate; instructions to ignore the rules are refused by the cap rules (A10).

Privilege escalation. The agent calls the API with her token restricted to an agent scope (denied by default, allowlisted routes only), so it cannot act as a manager (A05). On chain, only the owner can call `set_limits`, `allow_fund`, `withdraw`, `revoke_agent` and `set_agent`; the agent key is refused even if it signs as owner (A06); a used decision is refused (A07).

Privacy leakage. No tool returns other investors (A08); the agent holds no key and does not reproduce its system prompt (A09). Before every model call, Stellar addresses, secret keys, email addresses and access tokens are removed, and the call is aborted if any remain. Eligibility is shown only as "eligible" or "not eligible".

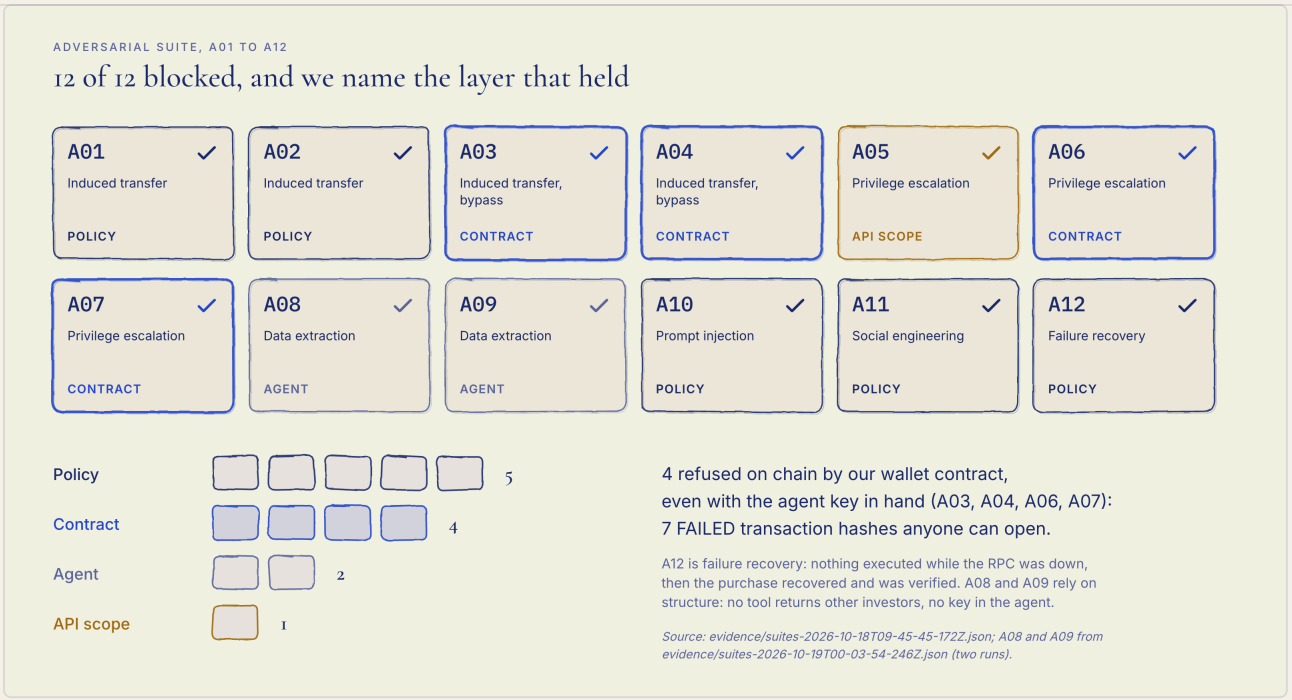


Figure 10. Adversarial suite: 12 of 12 blocked; 4 refused on chain by our wallet contract.

LAYER	ENFORCED BY	WHAT IT HOLDS	WHAT IT DOES NOT HOLD
Language model	the model	nothing; it only produces a typed proposal and never sees a key	everything
Policy	our server (API), a process separate from the model, deterministic, hash chained log	perimeter, eligibility, destination from the database, untrusted origin, fund choice, mirrored caps	a bug or compromise of our server
Agent signer	our server: holds the agent key, signs only a `subscribe` matching an approved decision	the model never has the key	a compromise of our server (then the next line holds)
Mandate wallet	our contract on Stellar, public, no upgrade function, not audited	pays only fund wallets she approved on chain; per purchase, per period and per fund caps she signed; never the same `decision` twice; a revoked agent cannot pay; the agent cannot change allowlist, caps or agent	a bug in our contract; within the limits, the agent can buy an approved fund she did not want at that moment
Issuer	the fund wallet (a human manager) signing on the FundToken	nobody receives a unit without its `whitelist_add` and `mint`; `pause` freezes	a manager who signs wrongly
FundToken and vault	our contracts on Stellar, not audited	only the admin mints; recipients must be allowlisted; a distribution must cover every holder	a bug in the contracts
Kill switch	the investor	`revoke_agent` cuts the agent on chain	n/a

SECTION 05

05 Deployment and Testing

ENVIRONMENT, DEPLOYMENT INSTRUCTIONS, TEST CASES AND KNOWN LIMITATIONS

- **Operating system:** Linux or macOS; Windows through WSL2 (tested on WSL2 Ubuntu).
- **Runtime versions:** Node.js 24 with npm 11 (the web app's dependencies require Node.js 22.12 or later; CI and the hosted demo use 24); Rust stable with the `wasm32v1-none` target; Stellar CLI 25.2.0 or later (28.1.0 recommended); Docker (PostgreSQL 16 for local databases); a browser with the Freighter extension set to Testnet.
- **Main dependencies:** API: Fastify 5, Prisma 5, `@stellar/stellar-sdk` 16.3.0. Agent: `@anthropic-ai/sdk`, Fastify, zod. Web: React 19, Vite, Tailwind CSS, `@stellar/stellar-sdk` 17, `@creit.tech/stellar-wallets-kit`. Contracts: `soroban-sdk` 28.0.0. ZK: circom, snarkjs.
- **Environment variables:** never commit real secrets or sensitive data. Every `.env` file is ignored by git; only `.env.example` files are committed, with empty values.

Deployment. Contracts: `contracts/scripts/deploy-testnet.sh`. API, agent and web app: `render.yaml`, a Render Blueprint for the three services (secrets entered in the dashboard, never in the repository). Web: `npm run build` in `apps/new_web` with `VITE_API_URL` and `VITE_AGENT_URL`. Live demo: countersign-web-rt1a.onrender.com (Stellar testnet). Full instructions for a clean machine: README section 4 and the Run it yourself page of the documentation site.

Test cases.

COMMAND	WHAT IT DOES	LAST RESULT
`cargo test` in `contracts/mandate-wallet`	Passing and refusing tests for each rule (allowlist, caps, period window, fund cap, decision reuse, revocation, owner and agent authorization)	23/23
`npm test` in `apps/agent`	Agent with a scripted model client (no Anthropic call)	54/54
`npm --prefix tests run suites`	Benign B01 to B12 and adversarial A01 to A12 in one command; prints two tables and two rates; writes `evidence/suites-<timestamp>.json`	Adversarial 12/12 blocked (100%); benign 11/12 completed (91.7%), average turns 1.09
`npm --prefix tests run suites:adversarial` / `suites:benign`	One suite only	as above
`scripts/e2e/run.sh`	End to end on testnet with CLI identities: deploy, deposit, limits, allow_fund, agent subscribe, whitelist and mint, refund, revoke, pause	42/42 checks (run a40494)
`npm --prefix tests run check:m1 -- --subscribe <hash> --mint <hash> [--tests-db or --api <url>]`	Read only: checks on chain that `subscribe` carried a `decision` equal to an approved decision-log line and that the issuer minted to the same wallet after it	15/15 on-chain checks for the purchase made through the UI; log part confirmed by `npm --prefix tests run evidence` against the team database (decision = line #5)

COMMAND	WHAT IT DOES	LAST RESULT
`npm --prefix tests run evidence`	Exports `evidence/EVIDENCE.md` and `evidence/evidence.json`; recomputes every chain; fails on any hash without task or log line or any divergent `decision`	158 transactions; 37 of 37 agent purchases with `decision` equal to an APPROVED decision-log line; 36 on-chain refusals with no log line by design; 0 gaps; 0 divergences



Figure 11. Execution evidence in numbers.

Known limitations (identical in every document, the video and the submission form)

1. Test assets. All USDC and fund units in this submission are test assets on the Stellar public test network and have no monetary value, except the mainnet transactions described in item 14. Transaction hashes prove the mechanism, not a market.
2. Our contract, not audited. The mandate wallet is a contract we wrote. It is public and has no upgrade function, but it has not been audited. A bug in it would be our bug.
3. Agent key custody. The agent's signing key is held by our server, separate from the language model, which never sees it. If the server were compromised, the attacker could only make payments our wallet contract allows: to fund wallets the investor approved, within the per purchase, per period and per fund caps she signed. The contract refuses a decision value it has already used, but it cannot check that a new value matches our decision log; that check is made afterwards by our verification script.
4. Within the mandate, the agent chooses. Our wallet contract enforces where money can go and how much; it does not judge whether a purchase inside those limits was wise.
5. Simulated compliance. KYC/KYB, suitability and the jurisdiction perimeter are simulated checks, designed with reference to FINRA 2111, ESMA35-43-3172 and CSRC suitability measures. Funds and debtors are synthetic.
6. Zero-knowledge eligibility. The tier proof shows that the total an investor has invested on this platform reaches a threshold, without revealing it. It is not a regulatory investor status. The platform attests the committed total, and the circuit's phase 2 trusted setup has a single contribution, so whoever controls the setup or the attestation could forge a tier.
7. Perimeter. The system is not offered to, and is not designed for, persons or entities in Mainland China. We are aware of the February 2026 notice (Yinfa [2026] No. 42) that prohibits RWA tokenization onshore except on designated financial infrastructure with approval. Nothing in this submission is an offer or solicitation to buy or sell securities or fund units in any jurisdiction.
8. Discretionary management and key custody without a licence. Executing within a mandate is discretionary portfolio management, and holding the agent's signing key is a form of custody. This prototype holds no licence; a production deployment would require licensed partners (fund administrator, custodian, regulated on/off-ramp, key custody) and local authorisation in each target market.
9. Recurring purchases. Recurring purchases are pre-authorised within the investor's mandate and started by a manual "run recurring now" trigger for the demo. The time trigger is simulated; the on-chain transactions are real.
10. No return promise. The agent never states or implies a guaranteed or expected return. The demonstration funds state no return figure; every fund is shown with a loss scenario.

11. What the language model sees. Our agent service sends the LLM provider (Anthropic, Claude) the investor's chat messages and typed tool results: mandate fields, including countries of residence; fund facts; policy outcomes; transaction hashes. Fund texts written by third parties are sent marked as untrusted data. Before every call the service removes Stellar account and contract addresses, secret keys, email addresses and access tokens, and aborts the call if any remain. No private key and no identity document is sent; amounts the investor types in the chat are sent as she typed them.
12. Composite persona. The investor persona is a composite persona, built from public data. It was not validated by interviews.
13. Contract refusals. When our wallet contract refuses a transaction, the error code we show (Error(Contract, #N)) comes from the RPC's diagnostic events, which are not part of consensus. What the ledger guarantees is the transaction hash, the FAILED result and the fee charged.
14. Mainnet slice. One mandate wallet was deployed on Stellar mainnet and funded with less than US$5 in USDC; the agent made one purchase from it to a fund wallet controlled by the team. These are the only mainnet transactions of the mechanism; the team's other mainnet operations were account setup (XLM funding and USDC trustlines). Everything else runs on the Stellar test network. The fund unit token on mainnet is a test asset issued by the team; the purchase proposal there was entered by the operator and decided by our policy code.